```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)

text = "hello world. hello rnn. hello numpy. "    # EXAM DAY: replace
with any short text
chars = sorted(set(text))
V = len(chars)                                    # vocabulary size
c2i = {c: i for i, c in enumerate(chars)}
i2c = {i: c for c, i in c2i.items()}
data = [c2i[c] for c in text]
print("Characters:", len(text), "| Vocabulary:", V)
```

```
Characters: 37 | Vocabulary: 14
```

```
H, T, lr = 32, 10, 0.1       # hidden size, sequence length per step,
learning rate

Wxh = np.random.randn(H, V) * 0.01     # input  -> hidden
Whh = np.random.randn(H, H) * 0.01     # hidden -> hidden (the
recurrence)
Why = np.random.randn(V, H) * 0.01     # hidden -> output
bh = np.zeros((H, 1))
by = np.zeros((V, 1))
params = [Wxh, Whh, Why, bh, by]

def forward_backward(inputs, targets, hprev):
    xs, hs, ps, loss = {}, {-1: hprev}, {}, 0

    # forward pass
    for t in range(len(inputs)):
        xs[t] = np.zeros((V, 1)); xs[t][inputs[t]] = 1
# one-hot input
        hs[t] = np.tanh(Wxh @ xs[t] + Whh @ hs[t - 1] + bh)
# new hidden state
        y = Whh_out = Why @ hs[t] + by
# scores for the next character
        ps[t] = np.exp(y - y.max()); ps[t] /= ps[t].sum()
# softmax
        loss += -np.log(ps[t][targets[t], 0])
# cross-entropy

    # backward pass (through time)
    dWxh, dWhh, dWhy = np.zeros_like(Wxh), np.zeros_like(Whh),
np.zeros_like(Why)
    dbh, dby, dhnext = np.zeros_like(bh), np.zeros_like(by),
np.zeros_like(hprev)
    for t in reversed(range(len(inputs))):
        dy = ps[t].copy(); dy[targets[t]] -= 1
        dWhy += dy @ hs[t].T;  dby += dy
```

```
        dh = Why.T @ dy + dhnext
        dhraw = (1 - hs[t] ** 2) * dh
# through tanh
        dbh += dhraw
        dWxh += dhraw @ xs[t].T
        dWhh += dhraw @ hs[t - 1].T
        dhnext = Whh.T @ dhraw
# gradient flows to the previous step

    grads = [dWxh, dWhh, dWhy, dbh, dby]
    for g in grads:
        np.clip(g, -5, 5, out=g)
# gradient clipping
    return loss, grads, hs[len(inputs) - 1]

inp, tgt, h0 = data[:T], data[1:T + 1], np.zeros((H, 1))
_, g, _ = forward_backward(inp, tgt, h0)
eps = 1e-5
for (i, j) in [(0, 0), (3, 5), (10, 20)]:
    Whh[i, j] += eps;      lp, _, _ = forward_backward(inp, tgt, h0)
    Whh[i, j] -= 2 * eps;  lm, _, _ = forward_backward(inp, tgt, h0)
    Whh[i, j] += eps
    print("numerical:", (lp - lm) / (2 * eps), "| backprop:", g[1][i,
j])      # the two should match
```

```
numerical: 0.0001244833569558068 | backprop: 0.0001244832561866895
numerical: -0.0001026585039198835 | backprop: -0.0001026585862578162923
numerical: 0.0003283281202470789 | backprop: 0.0003283283255484035
```

```
mem = [np.zeros_like(p) for p in params]
losses, p, hprev = [], 0, np.zeros((H, 1))

for step in range(2001):
    if p + T + 1 >= len(data):                  # reached the end of
the text: start over
        p, hprev = 0, np.zeros((H, 1))
    inputs, targets = data[p:p + T], data[p + 1:p + T + 1]

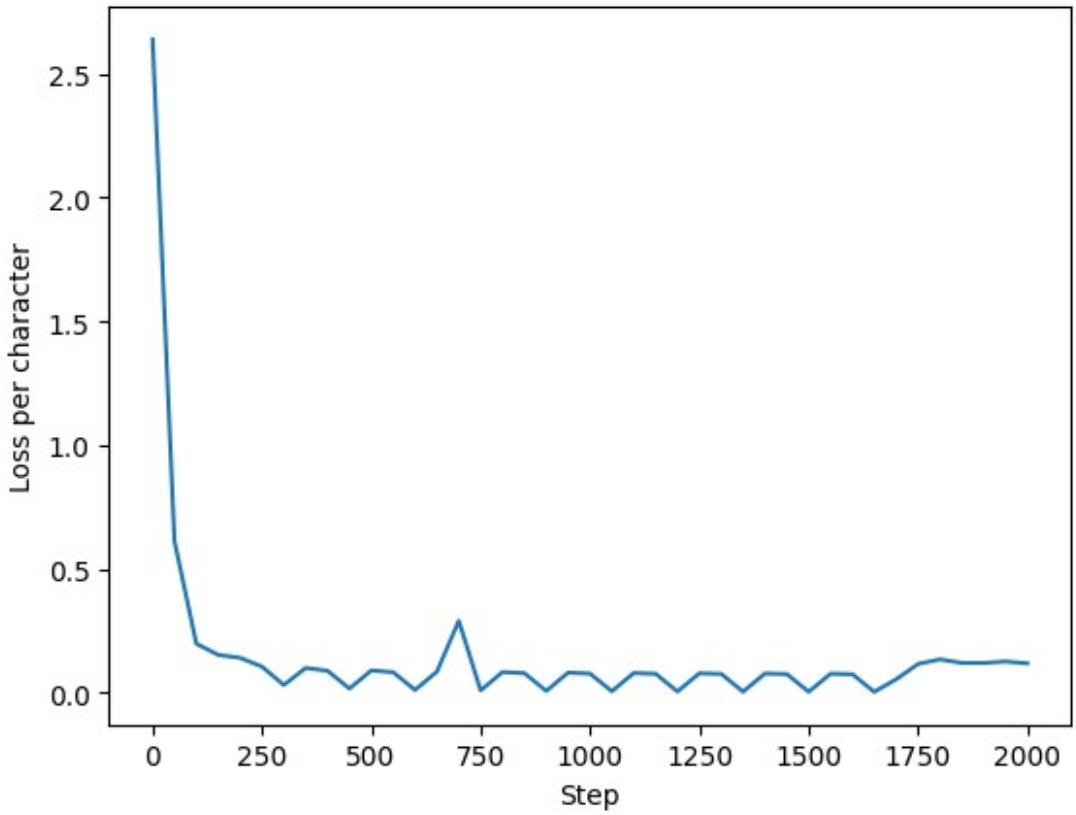
    loss, grads, hprev = forward_backward(inputs, targets, hprev)
    for prm, g, m in zip(params, grads, mem):
        m += g * g
        prm -= lr * g / np.sqrt(m + 1e-8)      # Adagrad update
    p += T

    if step % 50 == 0:
        losses.append(loss / T)
    if step % 400 == 0:
        print(step, loss / T)

plt.plot(range(0, 2001, 50), losses)
```

```
plt.xlabel("Step")
plt.ylabel("Loss per character")
plt.show()
```

```
0 2.6388146712758958
400 0.08923304314943516
800 0.08401011659677515
1200 0.005429989567815396
1600 0.07529583114146646
2000 0.11905609127903682
```




```
def sample(seed_char, n):
    h = np.zeros((H, 1))
    x = np.zeros((V, 1)); x[c2i[seed_char]] = 1
    out = [seed_char]
    for _ in range(n):
        h = np.tanh(Wxh @ x + Whh @ h + bh)
        y = Why @ h + by
        prob = np.exp(y - y.max()); prob /= prob.sum()
        idx = np.random.choice(V, p=prob.ravel())          # pick the
next character at random, weighted by the probabilities
        x = np.zeros((V, 1)); x[idx] = 1
        out.append(i2c[idx])
    return "".join(out)
```

```
print(sample("h", 60))
```

```
hello nnn. hello rnn. hello world. hello nnn. hello world. he
```